



UNIVERSITAS
GADJAH MADA

Configuration Drift pada Deployment Aplikasi Multi-Service di Kluster Kubernetes

Evaluasi Empiris Paradigma Manual dan GitOps
(Studi Kasus InvenioRDM)

Muhammad Argya Vityasy

NIM: 23/522547/PA/22475

Dosen Pembimbing: Dr. techn. Guntur Budi Herwanto, S.Kom., M.Cs.

Dosen Penguji 1: Dr. techn. Kabul Kurniawan, S.Kom., M.Cs.

Dosen Penguji 2: Aina Musdholifah, S.Kom., M.Kom., Ph.D.

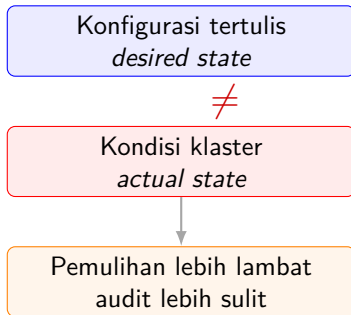
ugm.ac.id

LOCALLY ROOTED, GLOBALLY RESPECTED

Mengapa Masalah Ini Penting?

Skenario yang mudah terjadi

- Aplikasi berjalan di Kubernetes dengan banyak komponen.
- Operator melakukan perubahan cepat langsung di kluster.
- Dokumentasi konfigurasi tidak ikut berubah.
- Saat terjadi gangguan, tim tidak tahu kondisi mana yang benar.

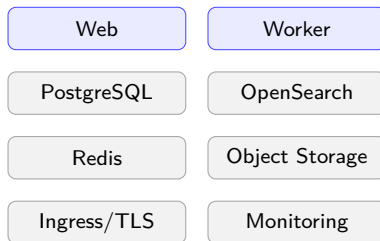


Inti masalah: bukan sekadar aplikasi rusak, tetapi hilangnya kepastian tentang state sistem.

Konteks: Deployment Multi-Service

InvenioRDM sebagai workload uji

- Platform repositori data penelitian berbasis web.
- Terdiri dari 16+ komponen dan dependensi.
- Mencakup aplikasi, database, cache, pencarian, storage, ingress, TLS, secret, monitoring, dan backup.



InvenioRDM bukan objek penelitian; ia dipakai sebagai subjek uji yang realistis untuk kelas workload multi-service.

Masalah Operasional: Configuration Drift

Drift terjadi ketika state aktual kluster menyimpang dari state yang dideklarasikan.

- Perubahan langsung pada kluster tidak masuk ke Git.
- Prosedur manual berbeda antar eksekusi.
- Tidak ada mekanisme deteksi otomatis.

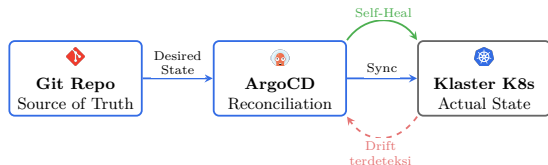


Konsekuensi penelitian: drift perlu diukur sebagai fenomena operasional, bukan hanya dijelaskan secara konseptual.

GitOps sebagai Mekanisme Kontrol

- Git menyimpan *desired state*.
- ArgoCD membandingkan Git dengan state aktual kluster.
- Jika terjadi deviasi, sistem dapat menandai *OutOfSync* dan melakukan *self-heal*.

Prinsip GitOps



Pertanyaan ilmiahnya bukan “apakah GitOps populer?”, melainkan seberapa berbeda respons operasionalnya saat drift terjadi.

Rumusan Masalah

Sejauh mana paradigma manajemen deployment yang berbeda, yaitu pendekatan manual menggunakan `kubectl` dan pendekatan GitOps menggunakan ArgoCD, menghasilkan perbedaan pada karakteristik operasional ketika menghadapi *configuration drift* yang terkontrol?

1. **R1 -- Konsistensi:** apakah deployment kembali sesuai dengan state yang diharapkan?
2. **R2 -- Waktu:** berapa lama drift terdeteksi, dipulihkan, dan akar masalah diidentifikasi?
3. **R3 -- Traceability:** seberapa lengkap perubahan dapat direkonstruksi?

Ruang Lingkup dan Tujuan

Ruang lingkup

- Satu kluster Kubernetes yang dikelola Rancher.
- Dua paradigma: manual dan GitOps/ArgoCD.
- Tujuh skenario drift terkontrol.
- InvenioRDM sebagai workload uji.

Di luar lingkup: multi-cluster, perbandingan semua alat GitOps, dan paradigma perantara Git tanpa agen reconciliation.

Tujuan

- Mengukur perbedaan konsistensi deployment.
- Mengukur waktu deteksi, pemulihan, dan identifikasi akar masalah.
- Mendeskripsikan traceability operasional kedua paradigma.

Manfaat dan Kontribusi

Teoritis

Bukti empiris terukur mengenai dampak drift pada paradigma deployment berbeda.

Metodologis

Protokol eksperimen yang dapat direplikasi: skenario, metrik, dan analisis statistik.

Praktis

Dasar keputusan operasional bagi pengelola klaster Kubernetes.

Kontribusi utama: bukan membuktikan GitOps selalu unggul, tetapi mengukur dampak operasional pilihan paradigma deployment.

Literatur: Apa yang Sudah Diketahui?

Sumber	Temuan Utama	Catatan
Shrestha & Ali (2024)	GitOps membantu deteksi drift	Dominan kualitatif
Kaggantinataraja (2025)	Pendekatan deklaratif lebih cepat dan aman	Variabel pengganggu belum terkontrol
Matubber (2025)	Reconciliation time dapat diukur	Tanpa pembanding manual
Paavola (2021)	ArgoCD cocok untuk banyak aplikasi	Komparatif deskriptif

Literatur mendukung potensi GitOps, tetapi belum cukup untuk menyimpulkan dampak kausal paradigma deployment.

Apa yang Membedakan Penelitian Ini?

Kelemahan studi sebelumnya

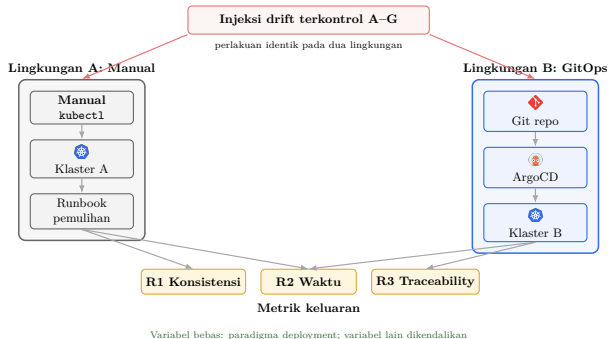
- Metrik drift belum selalu teroperasionalisasi.
- Tidak selalu ada *control group* manual.
- Variabel lingkungan dan operator belum dikendalikan.
- Analisis statistik formal masih terbatas.

Pembeda penelitian ini

- Desain eksperimental terkontrol.
- Drift identik pada dua lingkungan.
- Metrik R1--R3 didefinisikan sebelum eksperimen.
- Uji komparatif dan ukuran efek.

Dengan demikian, penelitian ini menutup celah validitas konstruk, validitas internal, metodologi, dan konteks workload akademis.

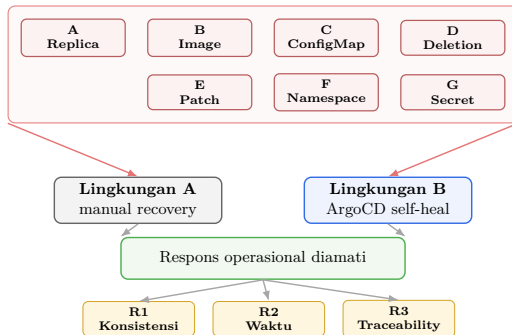
Gambar 3.1: Arsitektur Eksperimen



Satu variabel independen: paradigma deployment. Kontrol: workload sama, drift sama, spesifikasi klaster sama, prosedur pengukuran sama.

Skenario Configuration Drift Terkontrol

Tujuh skenario drift yang sama diberikan ke dua paradigma



Setiap skenario diinjeksi secara identik pada lingkungan manual dan GitOps, lalu respons operasionalnya diukur.

Metrik Pengukuran dan Operasionalisasi

Metrik	Operasionalisasi	Output
R1 Konsistensi	Persentase resource yang cocok dengan desired state	Nilai % pada baseline, saat drift, dan setelah recovery
R2a Detection time	$t_d - t_0$	Waktu dari injeksi hingga drift terdeteksi
R2b Recovery time	$t_2 - t_0$	Waktu dari injeksi hingga state kembali baseline
R2c Root cause ID	$t_{rci} - t_d$	Waktu dari deteksi hingga penyebab teridentifikasi
R3 Traceability	Bukti Git history, audit log, dan diff	Deskripsi kelengkapan rekonstruksi perubahan

R1 dan R2 dibandingkan antar paradigma; R3 dideskripsikan karena sifat audit trail berbeda secara struktural.

Jumlah perintah/intervensi manual dicatat sebagai pengamatan pelengkap, bukan metrik utama terpisah.

Validitas dan Fairness Eksperimen

Ancaman	Mitigasi dalam rancangan
Operator tunggal	Recovery runbook dan timestamp otomatis
Efek pembelajaran	Urutan skenario diacak dan ada practice run
Instrumentasi berubah	Versi Kubernetes, ArgoCD, dan workload dikunci
Satu workload	InvenioRDM dipilih sebagai workload multi-service realistis
Satu alat GitOps	ArgoCD digunakan sebagai implementasi GitOps yang umum

Klaim penelitian dibatasi: hasil berlaku sebagai evaluasi empiris terkontrol pada konteks yang didefinisikan.

Hasil yang Mungkin dan Maknanya

GitOps lebih baik

Klaim industri memperoleh dukungan empiris pada konteks terkontrol.

Tidak berbeda bermakna

Asumsi bahwa GitOps otomatis unggul perlu dibatasi.

Manual unggul pada metrik tertentu

Penelitian menunjukkan batas praktis dari otomatisasi GitOps.

Apa pun hasilnya, kontribusi penelitian adalah bukti yang dapat diperiksa, bukan opini adopsi teknologi.

*“Penelitian ini mengukur bagaimana
pilihan paradigma deployment
mengubah respons operasional terhadap
configuration drift -- bukan
membuktikan bahwa satu teknologi
selalu menang.”*



UNIVERSITAS
GADJAH MADA

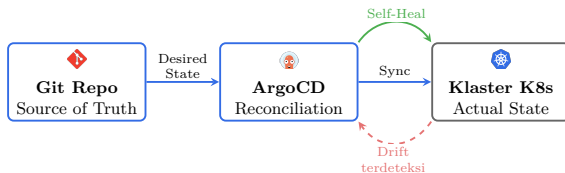
LOCALLY ROOTED, GLOBALLY RESPECTED

ugm.ac.id

Backup: GitOps -- Prinsip dan Mekanisme

- Git sebagai *single source of truth*
- Konfigurasi dideklarasikan secara deklaratif
- Agen *software* melakukan reconciliation
- Drift menjadi observable dan dapat dipulihkan otomatis

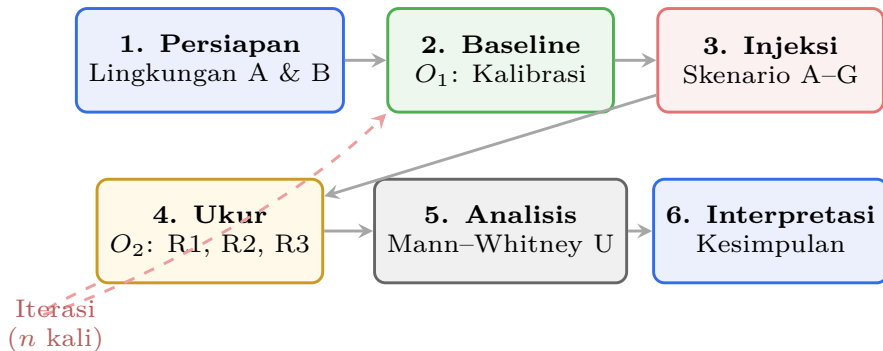
Prinsip GitOps



Backup: Detail Skenario Drift

ID	Skenario	Contoh perintah
A	Perubahan replika	<code>kubectl scale deployment --replicas=5</code>
B	Perubahan image	<code>kubectl edit deployment</code>
C	Perubahan ConfigMap	<code>kubectl edit configmap</code>
D	Resource deletion	<code>kubectl delete deployment</code>
E	Patch manual	<code>kubectl patch</code>
F	Namespace salah	<code>kubectl apply -f ... -n wrong-ns</code>
G	Perubahan Secret	<code>kubectl patch secret</code>

Backup: Bagan Alir Eksperimen



Tahapan mengikuti pre-test – treatment – post-test: baseline, injeksi drift, pengukuran, analisis, interpretasi.

Backup: Analisis Statistik dan Ukuran Sampel

Analisis

- Statistik deskriptif per metrik.
- Mann--Whitney U, $\alpha = 0,05$.
- Ukuran efek: Cohen's d atau Cliff's δ .
- R3 dianalisis secara deskriptif.

Pengulangan

- R1: $n = 5$ per skenario per lingkungan.
- R2: $n = 10$ per skenario per lingkungan.
- R3: dokumentasi bukti audit dan Git history.

Backup: Ancaman terhadap Validitas (Detail)

Validitas internal

- Seleksi lingkungan
- Operator tunggal
- Efek pembelajaran
- Perubahan instrumentasi

Validitas konstruk

- Definisi drift dibatasi tujuh skenario
- Traceability bersifat deskriptif

Mitigasi

- Klaster dan workload disamakan
- Runbook dan timestamp otomatis
- Urutan skenario diacak
- Versi software dikunci

Validitas eksternal

- Hasil dibatasi pada konteks workload dan klaster yang dijelaskan.